Tools and Workflow

Installing the toolchain, building a lesson,
and putting it on the board

The companion to *From Gates to a RISC-V CPU*.
That book explains the circuits; this one runs them.

Lattice ECP5 `LFE5U-85F` · Radiona ULX3S
Yosys · nextpnr · Project Trellis · Icarus Verilog · Verilator

Version 1.0 · 13 November 2026

This document is licensed under Creative Commons Attribution 4.0 International.
The accompanying code is licensed under the MIT License.
Copyright © 2026 The Open FPGA Course.

Contents

Installing 1
- What the installer does 1
- What `make check` reports 1
- Editor setup 1
- Icestudio, for the visual path 2

The tools, and the people who built them 3
- The chain, in order 3
- Who wrote them 3
- Why an open toolchain matters here 4
- Licences 4

Running a lesson 5
- The shape of a lesson 5
- The verbs 5
- From the project root, without changing directory 6
- Across the whole course 7
- The editor's task list 7
- The order to do things in 7
- On the exercises 8

Getting a design onto the board 9
- Is the board there? 9
- Build, then load 9
- Talking to it over serial 9
- Software, where a lesson has software 10
- Proofs, where a lesson proves something 10
- When something goes wrong 10

The board, pin by pin 11
- Clock 11
- LEDs 11
- Buttons 11
- Slide switches 11
- OLED — SSD1331, 96 × 64, 16-bit colour 11
- USB serial — FTDI FT231X 11
- Everything else 12
- Part facts worth remembering 12

Driving the tools directly 13
- Reading a resource report 13
- Which simulator when 13
- Tools written for this course 14
- Reproducing the numbers in the course book 14

Installing

One command, about a gigabyte, and nothing system-wide. Deleting the `toolchain/` directory undoes all of it.

```
git clone <this repository> && cd ULX3S
tools/install.sh
make check
```

`make check` verifies that every tool runs, then simulates lesson 00 as a smoke test. It ends with `Ready.`

What the installer does

`tools/install.sh` detects your platform, asks GitHub for the newest OSS CAD Suite release rather than pinning a date that rots, unpacks it into `toolchain/`, and on macOS clears the quarantine flags that would otherwise make every tool prompt for confirmation.

flag	adds	needed by
none	yosys, nextpnr-ecp5, Project Trellis, Icarus Verilog, Verilator, GTKWave, openFPGALoader, SymbiYosys	lessons 00–14
`--with-riscv`	a RISC-V bare-metal C compiler	lesson 15 onward
`--with-docs`	mdBook, for the HTML version of the lessons	optional
`--all`	all of the above	

Supported platforms are macOS (Intel and Apple Silicon) and Linux (x86-64 and ARM64). On anything else the script says so, and says where to put a toolchain you build yourself.

What `make check` reports

✓ `iverilog`	simulation
✓ `yosys`	synthesis
✓ `nextpnr-ecp5`	place and route
✓ `ecppack`	bitstream packing
✓ `verilator`	linting
✓ `gtkwave`	waveforms
✓ `dot`	schematics
✓ `make sim`	`PASS: 8 toggles, period 256 clks`

A missing *optional* tool is a dash rather than a cross, and the check still passes: `openFPGALoader` matters only with a board attached, `sby` only in lesson 14, a RISC-V compiler only from lesson 15.

Editor setup

Any editor works — the Makefiles are the interface. For VS Code the repository ships a complete configuration, and the one thing that matters is how you launch it:

```
tools/code.sh
```

Watch out. Editor extensions inherit the environment of whatever started the editor, and a desktop launcher gives them a bare `PATH`. Started from the Dock, the Verilog extension cannot find `verilator` and linting

silently does nothing — a failure that looks exactly like a broken extension. `tools/code.sh` puts `toolchain/` on `PATH` first. Open the **folder**, not a file, or `.vscode/tasks.json` never loads.

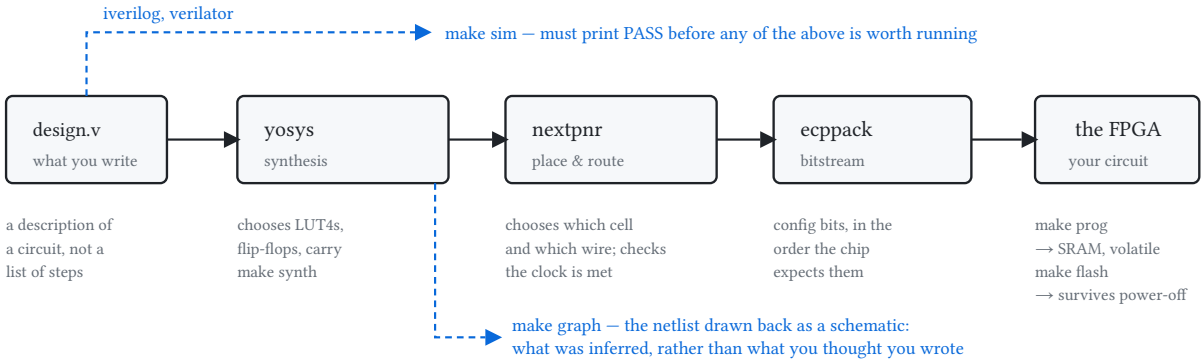
The workspace then offers one task per lesson, so running a lesson is **Terminal → Run Task → ▶ `deploy 00 blinky`** — two clicks, and it does not matter which file is open. VS Code tasks can only read the *focused editor tab* nothing lets them see what is selected in the file tree, which is why the lessons are enumerated as tasks instead.

Icestudio, for the visual path

Icestudio is a separate download from icestudio.io. Set the board to **ULX3S-85F** once in the setup wizard and the lessons' `.ice` projects open and build with no further configuration. It brings its own copy of the toolchain, so the two paths do not share `toolchain/` — redundant on disk, harmless otherwise.

The tools, and the people who built them

The chain, in order



Each stage is a translation, not a compilation to instructions: text, then a netlist of cells, then a placed and routed design, then configuration bits. The chip does not run the bitstream — it becomes what the bitstream describes.

From a Verilog file to a configured chip. Four programs, each solving a different problem, and each failing in its own characteristic way.

stage	what it turns into what
`yosys`	Verilog into a netlist of this chip's primitives. Goes wrong by inferring a latch you did not intend, or a hardware multiplier where you expected shifts
`nextpnr-ecp5`	that netlist into a choice of which physical cell and which physical wire, then checks the clock against the result. Goes wrong by not fitting, or by not meeting 25 MHz
`ecppack`	the placed design into a bitstream the chip will accept. Rarely goes wrong; by here the hard decisions have been made
`openFPGALoader`	a bitstream into the board, over the same USB cable that carries the serial port. Goes wrong by finding no device — usually the wrong USB socket or a charge-only cable

Alongside that path, and ahead of it, sit `iverilog` for simulation and `verilator` for linting.

Who wrote them

None of this course would exist without a long chain of mostly volunteer work, much of it given away.

Icarus Verilog	Simulation — the `make sim` in every lesson. **Stephen Williams**, since 1998: one of the oldest free Verilog implementations, and still the most forgiving one to learn in.
Verilator	The strict linter, and a very fast simulator. **Wilson Snyder**. Every `make lint` here is Verilator, and it caught real bugs while this course was written — a register driven from two `always` blocks, and a width truncation a passing test had hidden.
Yosys	Synthesis: Verilog in, a netlist of ECP5 primitives out. **Claire Wolf**, now YosysHQ. Every resource measurement in the course book is `yosys … stat`; every schematic is `yosys show`.
nextpnr	Place and route, and timing analysis. **gatecat** (formerly David Shah) with many others. Every maximum-frequency figure comes from here.
Project Trellis	The ECP5 bitstream documentation — the work that makes all of the above possible on this chip. Also **gatecat**'s, with contributors. `ecppack` and the PLL calculator `ecppll` are from this project.
GTKWave	Waveform viewing, by **Tony Bybell**. The `make wave` of every lesson.
openFPGALoader	Getting the bitstream onto the board, across a remarkable range of cables and boards. **Gwenhael Goavec-Merou**.
SymbiYosys	Formal verification, used in lesson 14. YosysHQ.

OSS CAD Suite	All of the above, built nightly for every platform as one archive, by YosysHQ. `tools/install.sh` downloads exactly this, which is why setup is one command instead of an afternoon.
Icestudio, apio	The visual path: a block-diagram editor and the build system under it, by **Jesús Arroyo Torrens** and the **FPGAwars** community. apio wraps the same yosys and nextpnr, so the two paths are two front ends onto one toolchain.
ULX3S	Open hardware from **Radiona.org**, the Zagreb makerspace, with **Goran Mahovlić** and **Davor Jadrijević** ("emard"), whose collection of example designs — SDRAM controllers, video, emulators of older machines — is worth reading once the course is finished.
RISC-V	Designed at UC Berkeley by **Krste Asanović**, **Andrew Waterman**, **Yunsup Lee** and **David Patterson**, and released as an open standard — which is why a course can end with "now build the processor" rather than "now read about one".
riscv-formal	A formal specification of RV32I an implementation can be checked against, by **Claire Wolf**. Lesson 14 uses it to prove the lesson 13 core correct rather than merely tested.

Project IceStorm — Claire Wolf's documentation of the Lattice iCE40 bitstream, published in 2015 — is the root of this tree. It demonstrated that a modern FPGA's bitstream could be understood well enough to target with free tools, and Project Trellis then did the same for the much larger ECP5. A commercial toolchain is a licence-managed install from a vendor with a commercial interest in the chips it targets; the equivalent used here was largely written by individuals, is readable end to end, and asks nothing. If this course is useful to you, those projects are where the gratitude should go — and most of them take contributions from people whose first design was a blinking LED not long before.

Why an open toolchain matters here

- **Nothing expires and nothing phones home.** No licence server, no registration, no node-locked key that stops working when you change laptop.
- **You can read the tools.** When yosys does something surprising — and the course documents several cases — the source is there. The schematic `make graph` produces is the tool showing you its own intermediate representation.
- **It runs anywhere.** One download of about a gigabyte, no administrator rights, removed by deleting a directory.

Licences

Code	MIT. The Verilog, the tools, the Makefiles and the assembly programs. Use them in anything, commercial or not; keep the copyright notice.
Prose	Creative Commons Attribution 4.0. The lessons, both books and the diagrams. Share, adapt, translate, teach from them — commercially if you like — giving credit and indicating changes.

One file is not this project's to license: `common/ulx3s_v20.lpf`, the board's pin constraints, which come from the ULX3S project and are included because the course cannot work without them. Everything else is original to the course.

Running a lesson

The shape of a lesson

Every lesson is a directory holding its own source, testbench, Makefile and written lesson:

```
13-rv32i-single/
  README.md        the lesson
  cpu.v            synthesisable source
  tb_cpu.v         testbench — always prints PASS or FAIL
  test.s           an assembly program, where the lesson needs one
  Makefile         four lines; the rules live in common/lesson.mk
  diagrams/        timing diagram sources
  icestudio/       the visual path: a spec and the project built from it
```

A lesson's Makefile declares only what is different about it and includes the shared rules:

```
TOP = blink          # synthesisable top module, in blink.v
TB  = tb_blink       # testbench module, in tb_blink.v
include ../common/lesson.mk
```

`common/lesson.mk` is the whole build system — about 90 commented lines, worth reading once.

The verbs

Every lesson understands the same ones. Run them from inside a lesson folder.

verb	tool	what it does
`make sim`	iverilog + vvp	compile source and testbench, run it. **Must print `PASS`**
`make wave`	GTKWave	open the VCD this lesson produced
`make lint`	Verilator `-Wall`	width mismatches, inferred latches, multiple drivers
`make graph`	yosys + graphviz	render the design as an SVG schematic
`make synth`	yosys `synth_ecp5`	resource report: LUT4s, flip-flops, block RAM, carry cells
`make bit`	yosys → nextpnr → ecppack	a bitstream, with timing closure
`make prog`	openFPGALoader	load into FPGA SRAM — volatile, for iterating
`make flash`	openFPGALoader `-f`	write to SPI flash — survives power-off
`make clean`		delete build products

Options the verbs take

`make synth` reports the design as built, which is flattened — so a submodule's own cost is only visible by making it the top. `M` does that, and `P` overrides one of its parameters. Every per-module number in the course book was produced this way, and the lesson always says which command produced which:

```
make synth                        # the lesson's top, as built
make synth M=uart_rx              # just the receiver
make synth M=adder_ripple P="W 4" # that module at a different width
```

The rest are variables a lesson's own Makefile overrides, listed here because reading an unfamiliar lesson's Makefile is easier when the knobs have names:

variable	default	what it is for
`SRCS`	`$(TOP).v`	the synthesisable sources
`TB_SRCS`	`$(TB).v`	the testbench sources
`SIM_SRCS`	`$(SRCS)`	what to *simulate*, when that differs. A lesson instantiating a hard block — the PLL, a block RAM primitive, an I/O buffer — cannot simulate it: there is no behavioural model in iverilog. Such a lesson names only the simulatable part here and verifies the rest structurally with `make bit`. Lesson 04 is the first to need it
`DATA`	empty	memory images read by `$readmemh`. These are read at *elaboration* time, by iverilog and by yosys, so they are build inputs exactly like a `.v` file. A lesson that leaves them out quietly simulates and flashes a **stale** program after you rebuild the software — which lesson 15 did, reporting nothing to be done while `firmware.hex` was newer than the bitstream
`LINT_SRCS`, `LINT_TOP`	from `SIM_SRCS`	what Verilator sees
`VCD`	`$(TOP).vcd`	the waveform file `make wave` opens
`DEVICE`, `PACKAGE`, `BOARD`	`--85k`, `CABGA381`, `ulx3s`	the part and the programmer target
`FREQ`	25	the clock nextpnr must prove the design closes at

Two Verilator warnings are waived course-wide and both are deliberate. `DECLFILENAME` enforces one module per file — right for a large project, wrong for a lesson where three related ten-line modules read better grouped in one file. `PROCASSINIT` objects to `reg q = 0;` on a signal that is also assigned procedurally, which the course does everywhere on purpose: the ECP5 loads flip-flop initial values straight from the bitstream, so a register really does start at a known value with no reset logic. Not every target honours that, which is why the lessons say so in prose rather than pretending it is portable. Every other `-Wall` warning is an error — a linter trained to cry wolf is a linter you stop reading.

From the project root, without changing directory

`L` is the lesson number. `5`, `05` and `05-fsm` all find the same lesson:

```
make list              # which lesson is which number
make deploy L=0        # build lesson 00 and load it onto the board
make sim    L=5        # simulate lesson 05
make wave   L=3        # open lesson 03's waveform
```

Every per-lesson verb works this way — `sim bit prog flash lint synth graph wave clean deploy`. With no `L`, the Makefile says which lessons exist rather than failing cryptically. This exists because remembering which editor tab is focused, or which directory a terminal is in, is friction nobody needs.

Across the whole course

`make check`	verify the toolchain and smoke-test lesson 00
`make sim-all`	every lesson's testbench — the regression suite
`make lint-all`	Verilator across every lesson
`make graph-all`	render every lesson as a schematic
`make ice`	regenerate the Icestudio projects from their specs
`make icecheck`	compile-check every Icestudio project, without the GUI
`make tasks`	regenerate the editor's per-lesson task list
`make docs`	build the HTML version of the lessons
`make docs-serve`	serve that at `http://localhost:3000`
`make pdf`	build both books: the course and this document
`make pdf-watch`	rebuild them on every save
`make clean-all`	remove build products everywhere
`make help`	list the above

Keeping the whole course green

```
make sim-all
```

A later lesson must never break an earlier one, so the whole course is a regression suite. When a module is promoted into `common/rtl/` — as the adder and the debouncer are, once a second lesson needs them — this is what catches a change that breaks a downstream lesson. Worth running before every commit; it ends with `ALL LESSONS PASS` or names what failed.

The editor's task list

Optional, and entirely equivalent to the `L=` commands above. **Terminal → Run Task…** lists every lesson by name, deploy entries first; clicking one builds it and loads it onto the board, with nothing selected beforehand and no regard for which file is open.

keys	action	which lesson
⌘⇧B	`make sim` — run the testbench	the open file's
⌘⌥R	the task picker — everything	you choose
⌘⌥U	build **and** deploy	asks
⌘⌥B	build a bitstream only	asks
⌘⌥P	deploy an existing bitstream	asks
⌘⌥L	`make lint` — Verilator, strict	the open file's
⌘⌥G	`make wave` — GTKWave	the open file's
⌘⌥Y	is the board connected?	—
⌘⌥T	stop the running task	—

"The open file's" means the task runs in the directory of whichever editor tab is focused. With a root-level README focused there is no lesson, and the Makefile says so.

The order to do things in

1. **Read the lesson's idea** before looking at any code. Most lessons are about one idea and the code is its consequence.
2. **`make sim`.** It must print `PASS`. If it does not, nothing else is worth doing yet.
3. **`make wave`**, and look at the three signals the lesson tells you to look at. A waveform with forty signals in it is not useful until someone says which three matter.

4. **`make synth`**, and read the number. Then try to account for it. This is the habit that separates "it works" from "I know what I built".
5. **`make graph`**, at least once per lesson. Seeing what the synthesiser inferred from what you wrote is the fastest way to find the gap between the two — an `always` block you thought was a register that became a latch, a comparison you thought was 4 bits wide that became 32.
6. **`make prog`**, where the lesson has a board demo.
7. **The exercises.** Most break something deliberately. Predict what will happen before you run it.

On the exercises

They are not homework. Each is chosen because it teaches something by failing in an instructive way, and each states what you should observe when it works. Three kinds recur: **break it and watch** ("change `⩽` to `=` and run `make sim`" — the failure is the lesson), **predict then measure** ("how many flip-flops will this use? now run `make synth`"), and **extend it** ("add a parity bit"), which are the ones that take an evening and the ones worth doing. Worked solutions ship only where an answer is genuinely hard to self-check; lesson 01 has a set, most lessons do not need one.

Getting a design onto the board

Is the board there?

The ULX3S appears as an FTDI device, vendor `0403`, product `6015`. One USB cable carries both the programmer and the serial port.

```
toolchain/oss-cad-suite/bin/openFPGALoader --detect -b ulx3s
```

A working board answers with its JTAG identity:

```
idcode 0x41113043
manufacturer lattice
family ECP5
model  LFE5U-85
```

Watch out. The ULX3S has **two** USB connectors and only one of them is wired to the FPGA's programmer. If `--detect` finds nothing, try the other socket before suspecting anything else — and check the cable is a data cable rather than a charge-only one, which is the other common cause of a board that appears dead.

On macOS this works with no driver installation. On Linux you need udev rules so that programming does not require root: use the rules file shipped with openFPGALoader, then `sudo udevadm control --reload`.

Build, then load

```
make bit       # yosys → nextpnr → ecppack, with timing closure
make prog      # into the FPGA's configuration SRAM
make flash     # into the SPI flash beside it
```

Or, from the project root, both steps at once — the command you will use most:

```
make deploy L=0
```

`make prog`	straight into the FPGA's configuration SRAM. Takes about a second, and is gone the moment the board loses power. This is what you use while iterating
`make flash`	into the SPI flash chip beside the FPGA, from which the FPGA reloads itself at every power-on. This is what you use when the design is finished

The ECP5 holds its configuration in on-chip SRAM, which is why there are two commands rather than one. An FPGA that has not been configured does nothing whatsoever — it is not running an idle loop; there is no loop.

Watch out. `make bit` reporting "Nothing to be done" means the bitstream is newer than its sources, so there is genuinely nothing to rebuild. If you believe otherwise, the usual cause is a build input the lesson did not declare — a `$readmemh` image, most often. `make clean` forces it.

Talking to it over serial

From lesson 09 onward a design can print. Lessons 15 and 17 add `make console`, which tells you the port and the incantation rather than pretending to be a terminal emulator:

```
make console
#   macOS:  screen /dev/cu.usbserial-* 115200    (quit: ctrl-a k y)
#   Linux:  picocom -b 115200 /dev/ttyUSB0       (quit: ctrl-a ctrl-x)
```

Two things about this cost an evening each if you meet them the hard way.

Use `/dev/cu.*`, not `/dev/tty.*`. On macOS the `tty` device blocks on open until carrier detect is asserted, which an FTDI cable never does. The open simply hangs, with no error to search for.

Quit the console before `make prog`. The ULX3S puts JTAG and this UART on one FT231X, so openFPGA-Loader cannot claim the chip while a terminal holds the port. It fails with `unable to claim usb device`.

Software, where a lesson has software

Lessons 15 and 17 run compiled C, so they add a software half to the same Makefile:

`make firmware`	`riscv64-elf-gcc` → `objcopy` → `tools/bin2hex.py`, producing `firmware.hex`
`make size`	where the program went — `.text` against `.bss`, and the memory it has to fit in
`make dis`	the program as the CPU sees it. Worth doing once: every instruction in there is one your decoder has to recognise
`make console`	the serial incantation, above

`firmware.hex` is checked in, and `make clean` leaves it alone, so a reader with no RISC-V compiler can still simulate, synthesise and flash those lessons — they just cannot change the program. `RVPREFIX` overrides the compiler prefix; `MEM_WORDS` the memory size the image is padded to.

Lesson 13 assembles rather than compiles, with the course's own assembler:

```
make prog-asm      # tools/rvasm.py test.s → test.hex, and a listing
make check-asm     # cross-validate that assembler against GNU as
```

Proofs, where a lesson proves something

Lesson 14 has no design of its own — it proves lesson 10's ALU and lesson 11's register file — so its `make sim` runs the proofs, which is what puts formal verification into the course-wide regression suite:

`make sim`	the quick proofs, as part of `make sim-all`
`make formal`	everything, including the register file (about a minute)
`make formal-broken`	a deliberately broken ALU, so the solver produces a counterexample
`make formal-vacuous`	a proof that passes and is worthless, with the cover that shows it
`make wave`	the counterexample trace from the broken proof

`lint`, `synth`, `bit` and `prog` in lesson 14 print a note pointing at lessons 10 and 11, where those designs are already checked.

When something goes wrong

`command not found`	the toolchain is missing — run `tools/install.sh`
macOS: "cannot be opened"	quarantine flags; `xattr -dr com.apple.quarantine toolchain/oss-cad-suite`
linting does nothing	the editor was launched without the toolchain on `PATH` — use `tools/code.sh`
`No rule to make target 'bit'`	`make` ran in the project root with no lesson named. Add `L=n`, or open a file inside a lesson first
`Nothing to be done for 'bit'`	the bitstream is newer than the source. `make clean` to force a rebuild
`openFPGALoader` finds nothing	wrong USB socket, or a charge-only cable
`unable to claim usb device`	a serial console still holds the FT231X. Quit it, then program
"no extension found for verilog"	you pressed the editor's Run/Debug button. Verilog is not *executed* — it is simulated or synthesised. Use `make sim`
`nextpnr` fails timing	the design is too slow for the requested clock. Read the critical path it reports; lesson 04 is about exactly this
a serial console that hangs on open	`/dev/tty.*` on macOS. Use `/dev/cu.*`

The board, pin by pin

The authoritative constraint file is `common/ulx3s_v20.lpf`. The names below are the port names to use in a top-level module; the constraint file binds them to ECP5 balls. Icestudio uses the same friendly names on its canvas, so a block bound to `btn_1` and a Verilog port `btn[1]` land on the same ball.

Clock

`clk`	G2	25 MHz, free-running

Every lesson in this course is clocked by this and nothing else — one clock domain, no gated clocks, except lesson 04 which is about the exceptions.

LEDs

Active high: write 1 to light. `led_0` is the least significant bit of the `led[7:0]` port used throughout.

`led_7`	`led_6`	`led_5`	`led_4`	`led_3`	`led_2`	`led_1`	`led_0`
H3	E1	E2	D1	D2	C1	C2	B2

Buttons

`btn_0` is the power button and is **inverted** — it reads 1 when released. The rest are active high with pull-downs. All of them need debouncing, which is lesson 03.

`btn_0`	D6	power — inverted
`btn_1`	R1	FIRE1
`btn_2`	T1	FIRE2
`btn_3`	R18	UP
`btn_4`	V1	DOWN
`btn_5`	U1	LEFT
`btn_6`	H16	RIGHT

Slide switches

Static configuration input; no debouncing needed in practice.

`sw_0`	`sw_1`	`sw_2`	`sw_3`
E8	D8	D7	E7

OLED — SSD1331, 96 × 64, 16-bit colour

Four-wire SPI, write-only. `oled_dc` selects command (0) or data (1). Maximum clock about 6.6 MHz, which is why lesson 06 divides the 25 MHz clock by four.

`oled_clk`	P4	SPI clock, driven by the FPGA
`oled_mosi`	P3	data, most significant bit first
`oled_dc`	P1	command (0) or data (1) — a level, not a stream bit
`oled_resn`	P2	reset, active low; needs a real delay after release
`oled_csn`	N2	chip select, active low

USB serial — FTDI FT231X

Watch out. Named from the **FTDI's** point of view. The FPGA *drives* `ftdi_rxd` and *reads* `ftdi_txd`, so `output ftdi_rxd` and `input ftdi_txd` are correct and look backwards. `ftdi_txden` must be driven high to enable the FPGA's driver. This is the classic place to lose an afternoon.

`ftdi_rxd`	L4	the FTDI receives — the FPGA **transmits**
`ftdi_txd`	M1	the FTDI transmits — the FPGA **receives**
`ftdi_txden`	L3	drive high to enable
`ftdi_nrts`	M3	
`ftdi_ndtr`	N1	

Everything else

The board definition lists 184 pins in total: 32 MB of SDRAM (`sdram_*`, 16 bits wide), digital video out (`gpdi_*`), the audio resistor ladders (`audio_*`), the SD card (`sd_*`), SPI flash (`flash_*`), an ADC (`adc_*`), the ESP32 interface (`wifi_*`), and 28 differential GPIO pairs (`gp0–gp27` and `gn0–gn27`). See `common/ulx3s_v20.lpf` for the complete set.

Part facts worth remembering

FPGA	Lattice ECP5 `LFE5U-85F-6BG381C`
Package	381-ball caBGA
Speed grade	6
LUT4s	83,640
Flip-flops	83,640
Block RAM	208 × 18 Kbit, about 3.7 Mbit
18×18 multipliers	156
PLLs	4
Clock	25 MHz
Programmer	FTDI FT231X — `openFPGALoader -b ulx3s`

Driving the tools directly

The `make` verbs are thin wrappers. When you want the tools themselves:

```
# simulate
iverilog -g2005 -Wall -o tb.vvp design.v tb_design.v && vvp tb.vvp

# strict lint
verilator --lint-only -Wall --top-module design design.v

# resource usage
yosys -p "read_verilog design.v; synth_ecp5 -top design; stat"

# resource usage at a different parameter value
yosys -p "read_verilog design.v; chparam -set W 32 design; \
          synth_ecp5 -top design; stat"

# schematic
yosys -p "read_verilog design.v; proc; opt_clean; \
          show -format dot -prefix d design"
dot -Tsvg d.dot -o d.svg

# place and route for the ULX3S 85F
nextpnr-ecp5 --85k --package CABGA381 --freq 25 \
  --json design.json --lpf common/ulx3s_v20.lpf --textcfg design.config
ecppack design.config design.bit

# program
openFPGALoader -b ulx3s design.bit        # SRAM
openFPGALoader -b ulx3s -f design.bit     # SPI flash
openFPGALoader --detect -b ulx3s          # is the board there?

# PLL parameters
ecppll -i 25 -o 100 --clkout1 40 -f pll.v
```

Reading a resource report

```
   LUT4              18     combinational logic, 4 inputs each
   TRELLIS_FF        26     flip-flops
   CCU2C             13     carry chain cells — TWO BITS EACH
   DP16KD             1     an 18 Kbit dual-port block RAM
   TRELLIS_DPR16X4    8     distributed RAM: 16 words x 4 bits, from LUTs
   PFUMX / L6MUX21    4     wide multiplexers, for functions over 4 inputs
   MULT18X18D         0     a hardware multiplier
```

The 85F has 83,640 LUTs, 83,640 flip-flops and 208 block RAMs, so a design using 34 cells is using about 0.04% of the part. Knowing roughly what things cost is the point: a 32-bit adder is 17 carry cells, a 32-bit barrel shifter about 667 LUTs, and a whole RISC-V CPU about 1,765.

Which simulator when

iverilog	the default. Fast to start, forgiving, good `$display`.
Verilator	much faster for long runs, but compiles to C++ and is strict about things iverilog tolerates. Used here mainly as a linter.
sby	not a simulator at all. Proves properties over *all* inputs up to a bound, instead of the handful your testbench thought of. Lesson 14.

Tools written for this course

`tools/install.sh`	set up a machine; no sudo, nothing system-wide
`tools/check.sh`	verify every tool runs, then smoke-test lesson 00
`tools/code.sh`	launch VS Code with the toolchain on `PATH`
`tools/rvasm.py`	an RV32I assembler — labels, pseudo-instructions, `--list`
`tools/check-asm.sh`	cross-validate that assembler against GNU `as`
`tools/bin2hex.py`	an ELF's `.bin` into the `$readmemh` image a lesson's memory loads
`tools/mkice.py`	generate Icestudio projects from lesson Verilog
`tools/icecheck.py`	rebuild and compile-check a generated .ice project
`tools/mktasks.py`	regenerate the editor's per-lesson tasks
`tools/mkwave.sh`	render WaveDrom timing diagrams to SVG
`tools/svg-flatten.py`	resolve CSS variables so diagrams print
`tools/mkdocs.sh`	build the HTML lessons
`tools/mkpdf.sh`	build both books

Reproducing the numbers in the course book

Every resource count and every frequency in the course book was produced by the commands printed beside it, on the setup below. Synthesis results are not universal constants: a different toolchain release can move a LUT4 count by a few percent, and a different device or speed grade moves the frequencies.

board	Radiona ULX3S, `ulx3s_v20.lpf` — the constraint file for board revisions v2.x.x and v3.0.x
device	Lattice ECP5 `LFE5U-85F-6BG381C` — 85F size grade, **speed grade 6**, 381-ball caBGA, commercial
clock	25 MHz, from the board crystal, constrained in the `.lpf` with `FREQUENCY PORT "clk_25mhz" 25 MHZ`
place & route	`nextpnr-ecp5 --85k --package CABGA381 --freq 25`
toolchain	OSS CAD Suite release `2026-11-10`
synthesis	Yosys 0.70+368 (`7cf88aca6`), `synth_ecp5`
p&r	nextpnr 0.11.1-89 (`445e2a65`)
bitstream	Project Trellis `ecppack` 1.4-88
simulation	Icarus Verilog 14.0 (devel)
linting	Verilator 5.055 devel

Resource numbers come from `make synth`, which runs `synth_ecp5` and `stat` — not from a placed design, so they do not include board wiring unless the quoted module is the lesson's top. Frequencies come from `make bit`, where nextpnr reports the routed design's maximum frequency for each clock.

A measured number without its conditions is an anecdote. If your own `make synth` disagrees with the book by a cell or two, the likely reason is a newer OSS CAD Suite — which is the right thing to trust, since it is the one that built your bitstream.